

Document Number: **P0671R2**, ISO/IEC JTC1 SC22 WG21
Audience: EWG
Date: 2018-05-07
Author: Axel Naumann (axel@cern.ch)

Self-explanatory Function Arguments

Table of Contents

0. [Don't think you had this discussion. Don't reject this without reading it.](#)
1. [The Feature in a Nutshell](#)
2. [Motivation](#)
3. [Alternatives](#)
4. [Counterarguments](#)
5. [Feature Details](#)
6. [Possible Extensions](#)
7. [Acknowledgments](#)
8. [Revision History](#)
9. [References](#)

0. Preamble

Please don't assume that you know what this paper is about (unless you've read [P0671R1](#)). Several people have invested time in discussing the feature, making it reasonable, and making it new (i.e. I am aware of past discussions, for instance [\[1\]](#)). This is *different* and I appreciate that you make up your mind based on what's written here. I am almost certain: you haven't seen this before.

1. The Feature in a Nutshell

One of my colleagues' and users' pet peeves with C++ is its lack of named parameters. A wonderful synopsis of pain is at [\[2\]](#). It's seen as a usability limitation caused by a C syntax anachronism. With many current languages, passing arguments can be done using the parameter index (C-style) or its name.

So here is what I suggest: allow the naming of arguments in regular (index-based) function calls, but ignore them from the standard's point of view — though not necessarily so from the implementers'. An example:

The call is completely equivalent to this traditional one:

But if the call looks like this:

then implementations could issue:

Wouldn't that be an incredibly nice and helpful compiler!

2. Lots of Motivation

Current function calls are tuple-style: a sequence of types. What C++ advocates for is classes with named members. But in current C++, member names are used much less than function parameters. How come then that we still think "a parameter index is all a call could possibly hope for"?

There are multiple usage patterns where code and humans suffer from this, in reality and on a daily basis. Here are the most relevant ones. If you know more, let me know!

Readability is maintainability

We all have functions that take two strings, or two ints, or two doubles - or a boolean. It's always awkward to read calls, especially when calling with literals because they don't have any semantic meaning attached. People came up with workarounds:

Apparently we need C-style comments to make C++ readable and maintainable [\[3\]](#). Or preprocessor-magic [\[4\]](#). Or the workaround called "use a struct and direct-initialize it". Or the "just-so-it-has-a-name" pattern of declared-once, used-once variables (see for instance [\[5\]](#), and an estimated count of 1900 occurrences in LLVM+clang for booleans alone):

Overload resolution

Overloading works like a charm when it works, and is a curse if the unsuspected happens. (If you question that statement, read the thread on ADL and [from the LWG reflector from February 2018](#).) C++ would be nice to its users if it offered argument names as a crosscheck that code author and compiler agree on the selected overload.

Call: what was the order again?

One of the major sources of errors we see in our code base is due to function calls with wrong parameter order, for instance caused by uncooperative interface evolution. The compiler might help in case of type mismatches, but very often it doesn't. Take the example from before: without looking it up, do you remember the name of the function from the introduction? You probably do: Gauss. Do you remember the name of the parameters? Likely. But do you remember their position? Compare

to . Err, wait - I meant
. With this proposal, a good implementation would tell you that you got the order wrong.

Names are far more significant than indices, which is why we use them so much in C++. This is not just cosmetics - this is an actual source of bugs.

3. Alternatives

IDEs

Use of an IDE can alleviate many of the pitfalls shown here when *writing* code. This does not make code more readable for humans. Maintenance cost is all about humans reading code.

Static Analysis

Existing static analysis (e.g. from clang-tidy [\[6\]](#)) can detect parameter mismatches in an invocation. It relies on parsing comments, which is inherently slow and fragile, because there is no syntax guarantee for comments: each tool will have its custom syntax requirements.

Coverity offers the checker. It detects a misplaced variable name passed as argument, matching the function's parameter names:

Evidently, this checker has limited reach and can easily produce false positives (see for instance [\[7\]](#)). And it's not the checker who is to blame here but C++.

The fact that these analysis checkers exist is a proof that there is demand, and that these checkers are inactive in mainstream compilers shows that they are difficult to get right (and fast) when based on C-style comments.

Attributes

One could deploy a new attribute:

This is syntactic sugar gone mad: yes it fulfills all technical requirements, but it fails at improving readability. Parameter naming is meant to make code more accessible for novices, for instance in tutorials; the syntax of attributes makes this impossible.

Designated Initializers

Designated initializers allow to give semantic meaning to parameters, by creating an aggregate class whose only purpose is to give names to parameters. This is a hack: the developer did not mean to pass an object; the intent was to pass parameters, but C++ does not allow parameter naming. This approach increases complexity (introducing the new aggregate type) for a novice-oriented feature - that's not going to succeed. And as a matter of fact it did not, for the many years that compilers support direct initialization and that developers ask for C++ support for naming parameters.

But this is not the real thing!

In many other languages, parameter naming allows to specify arguments in arbitrary order, skipping arbitrary defaulted parameters. This enables much more stable interfaces - but it would introduce an ABI dependence on names (see the discussion on R0 of this proposal). So yes, this is not the real thing, far from it. But it's addressing an issue that C and C++ call syntax have since their conception, without turning C++ functions inside out.

4. Counterarguments

"Parameter names are not stable"

C++ does not attach any meaning to a function declaration's parameter names. But in C++, the vast majority of functions that developers will invoke is declared exactly once, in a header file: developers will often not see the function's implementation. Even where redeclarations happen, many current coding conventions require the consistent use of parameter names: for coding conventions, consistency is paramount. And there is a checker for that: [\[8\]](#).

This means that this argument is missing the point: it is discussing on the language level, while this feature addresses a usability issue that does not have an impact on the language design (except for its syntax). A similar discussion took place in the reflection area, where function reflection is slated for satisfying several use cases by reflecting function parameter names [\[9\]](#).

"The standard library can't cope"

Users cannot rely on parameter names in current standard library implementations, but might try. Implementations can certainly detect this case, for instance thanks to their preprocessor-evading, uglified, reserved identifiers (). Future standard library interfaces might want to standardize parameter names, especially with the advent of the only proper solution to preprocessor mess in headers: modules.

"Not needed: C and C++ survived for decades without"

And then came languages that showed developers what they are missing in C++. And if this argument would be relevant, no further language evolution should be allowed - as a matter of fact: C++ would have been unnecessary because there was C.

We must not take decades of failure to respond to user demand as an excuse to continue to ignore them.

"Encourages APIs with many parameters"

A readability improvement in one part of C++ can mean that developers will invest that bonus to make code less readable in other places. We cannot force well-written code onto the world; this is neither our role nor within our power. What is within our role and power, and what I see as our duty, is to make C++ novice-friendly and maintainable. Acknowledging that this feature is a helpful simplification, and then turning this around as "and thus people can make things more complex" is an absurd perversion of the argument.

5. Feature Details

Syntax

In function calls, arguments can be named by separating an identifier with `&` from the argument's expression. The use of the identifier has no effect. And as a *Note* we will add that implementations could verify whether this identifier is used as parameter name in any of the redeclarations of the called function.

Again: this proposal does not introduce a new function type, changes nothing in function declarations, is a complete opt-in syntactic sugar for function call expressions. So is the use of names for accessing data members these days (instead of `*`), but wow are member names useful.

Possible diagnostic implementation wrt redeclarations and overridden functions

It would seem reasonable to issue no diagnostic if any declaration of the called function uses the parameter name for the argument it was specified for. If the function is virtual, parameter names used in base classes' declarations could be considered, too. The implementation cost is likely a (possibly lazily built - at least until argument names rule the world) set of known names for each parameter.

Effect on the standard

No effect on the existing standard is expected: this proposal does not change the normative behavior. It might have an effect on the future committee: if the world falls in love with this call syntax, agreeing on parameter names for post-modules "std2" might be a good idea. Until then, implementations would probably not warn about parameter name mismatches for functions in system headers.

6. Possible Extensions

These ideas are out of scope for this paper, but have been brought up during discussions.

"Use my name" attribute

A function declaration's parameters could be annotated through a new attribute, to require invocation through argument naming. A call that does not specify the parameter name would be ill-formed, just like a re-declaration of the function using different parameter names. This would give fairly "stable" names for parameters through the back door, while still allowing functions to stay regular functions.

As a further extension, parameters with default values could be skipped if the function is marked as "use my parameters' names".

Template Parameters

Named template arguments could be considered, and be it just for symmetry reasons with function arguments. Currently, the most common template argument kind is a type, which would likely not warrant naming the template arguments (because in C++, most types have names). This might change once `constexpr` values (and here especially literals) become common template parameters. Any decision here should probably wait until the schism between template meta programming and `constexpr`-value-based programming is settled.

7. Acknowledgments

Thanks to all the CERN and Fermilab folks who insisted that this matters and provided valuable feedback. Thanks to Ville for setting this paper onto the right track. Thanks to Alisdair for a very helpful review.

8. Revision History

Changes compared to revision 0 ([P0671R0](#) → [P0671R1](#))

Get the core features with a fairly unintrusive change: new syntax that is ignored for the standard, but enables compilers to emit diagnostics.

Changes compared to revision 1 ([P0671R1](#) → **this version**)

Change delimiter token to `[[`. Add sections "Alternatives", "Counterarguments", "Extensions".

9. References

1. [Named arguments](#) (N4172) by Ehsan Akhgari, Botond Ballo, and its [discussion notes](#) from Urbana-Champaign.
2. [Bring named parameters in modern C++](#) by Marco Arena (retrieved on 2017-06-13).
3. [Typical example of c-style comments to provide a name to an argument](#).
4. [Boost parameter library](#) (retrieved on 2017-06-13).
5. [Github: mongo/db/storage/wiredtiger/wiredtiger_recovery_unit.cpp](#), an example of named-parameters-through-local-vars (retrieved on 2018-05-06).
6. [clang-tidy: bugprone-argument-comment](#) (retrieved on 2018-05-06).
7. [Attachment to compiz bug 1101602](#) (retrieved on 2018-05-06), showing SWAPPED_ARGUMENTS at line 491 versus declararion at line 137.
8. [clang-tidy: readability-named-parameter](#) (retrieved on 2018-05-06).
9. [Function Reflection \(P0671R0\)](#) by Matus Chochlik, Axel Naumann, David Sankel.